

Universidade do Minho

Algoritmos e Estruturas de Dados

Engenharia de Telecomunicações e Informática

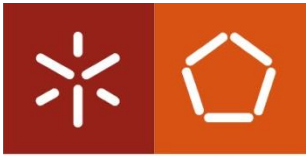
2022/2023

Telmo Adão

Departamento de Sistemas de Informação

Universidade do Minho

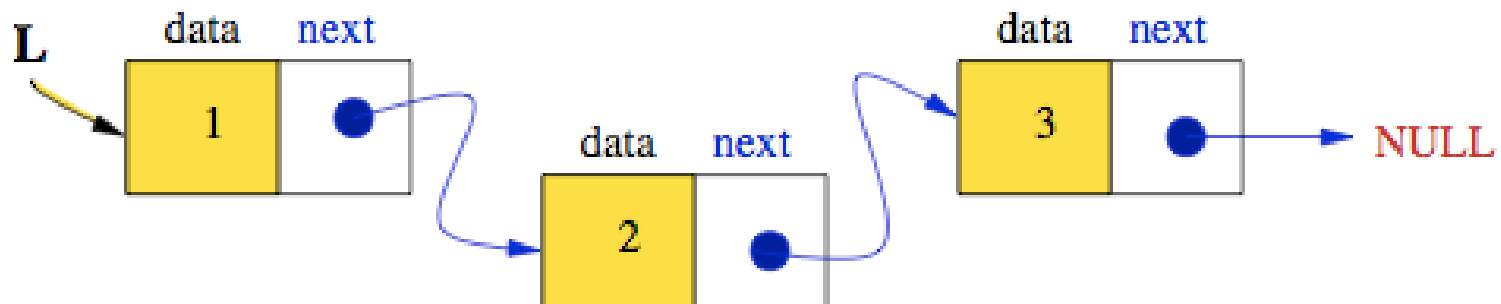
Slides baseados no material didático disponibilizado pelo Professor Luís Magalhães

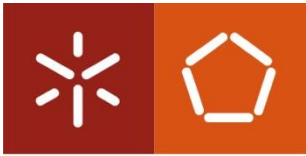


Listas Ligadas Simples

Universidade do Minho

- Uma lista ligada é uma estrutura de dados linear e dinâmica.
- Dados sequenciais, organizados por nodos
- Mais eficiente para um uso intensivo de inserções e remoções
- Menos eficiente para aceder a um dado elemento: $O(n)$
- Lista vazia, não tem nodos (0)

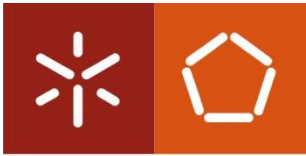




Universidade do Minho

Listas Ligadas Simples

- Cada nó ou elemento de uma lista encadeada irá possuir o valor do nó e o endereço do próximo nó.
- Numa lista ligada simples o último elemento aponta para NULL.
- Em C, a implementação de uma lista ligada passa pela definição de um tipo de dados para representar um nó que inclui os dados a armazenar e um apontador para o nó seguinte.
- Terá ainda um apontador para o início da lista



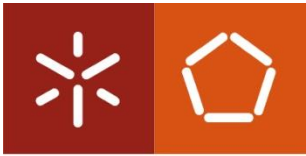
Listas Ligadas Simples

- Exemplo:

```
/* definição do tipo de dados a armazenar em cada nó */  
typedef int data_t;
```

```
/* definição de um nó, contendo os dados e o apontador  
para o próximo elemento */
```

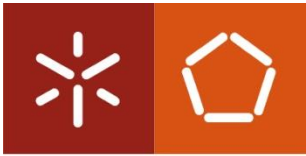
```
typedef struct node {  
    data_t data;  
    struct node* next;  
} node_t;  
node_t *lista;
```



Universidade do Minho

Listas Ligadas Simples

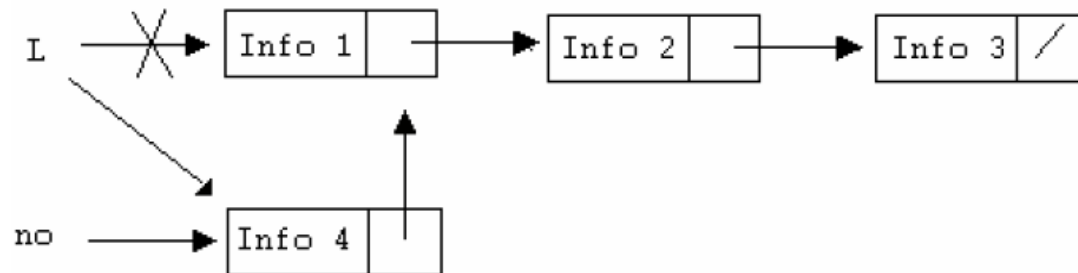
- Operações mais comuns:
 - Inserir elemento
 - Remover elemento
 - Percorrer lista
- Inserir elemento
 - Inserir no início
 - Inserir no fim
 - ... (podem existir outras variantes, por exemplo, inserir de forma ordenada)



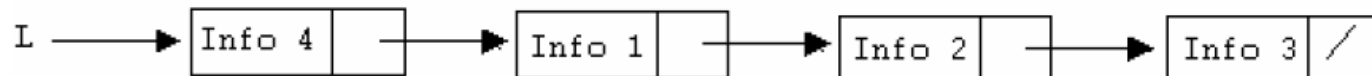
Listas Ligadas Simples

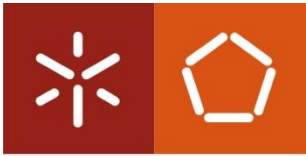
Universidade do Minho

- Inserir no início
 - Quando o nó é inserido no início da lista deve-se fazer o campo "próximo" deste nó apontar para o primeiro elemento da lista, e em seguida fazer a lista apontar para este nó recém-criado.



E então teremos:



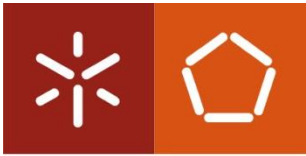


Universidade do Minho

Listas Ligadas Simples

- Inserir no início
 - Uma possível implementação em C

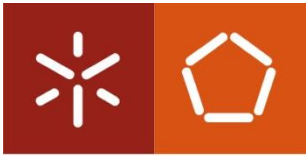
```
void inserirInicio(node_t **lista, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
    }  
    else{  
        elemento->next = *lista;  
        *lista = elemento;  
    }  
}
```



Universidade do Minho

Listas Ligadas Simples

- Inserir no fim
 - Quando o nó é inserido no fim da lista, deve-se fazer o campo "próximo" do último nó da lista apontar para o elemento que se deseja inserir. Para tanto, devemos percorrer a lista até encontrar o último elemento. Lembrando que o último elemento sempre aponta para NULL.

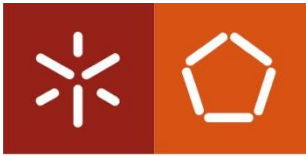


Listas Ligadas Simples

Universidade do Minho

- Inserir no fim

```
void inserirFim(node_t **lista, node_t *elemento){  
    node_t *aux;  
    aux = *lista;  
    if(*lista == NULL){  
        *lista = elemento;  
    }  
    else{  
        /* Percorre a lista até encontrar o último elemento */  
        while (aux->next != NULL){  
            aux = aux->next;  
        }  
        aux->next = elemento;  
    }  
}
```

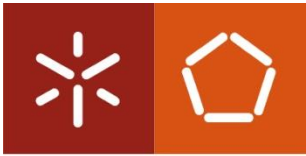


Universidade do Minho

Listas Ligadas Simples

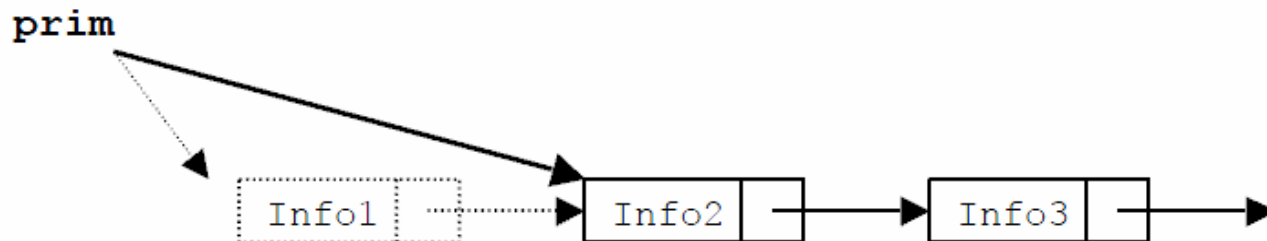
- Inserir no fim
 - Se mantivermos um apontador para o último elemento da lista, a inserção torna-se mais simples.

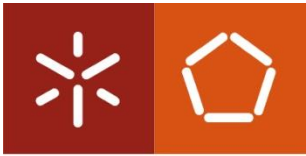
```
void inserirFim(node_t **lista, node_t **fim, node_t *elemento){  
    if(*lista == NULL){  
        *lista = elemento;  
        *fim=elemento;  
    }  
    else{  
        (*fim)->next = elemento;  
        *fim=elemento;  
    }  
}
```



Listas Ligadas Simples

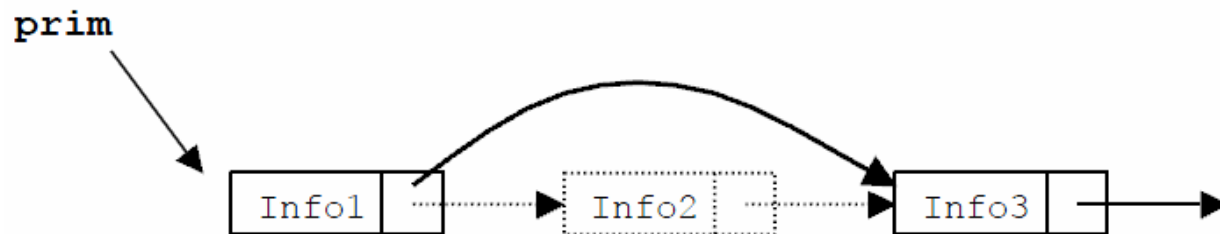
- Remover um elemento
 - A função para retirar um elemento da lista é mais complexa.
 - Se descobrirmos que o elemento a ser retirado é o primeiro da lista, devemos fazer com que o novo valor da lista passe a ser o apontador para o segundo elemento, e então podemos libertar o espaço alocado para o elemento que queremos retirar.

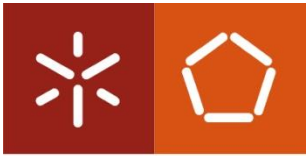




Listas Ligadas Simples

- Remover um elemento
 - Se o elemento a ser removido estiver no meio da lista, devemos fazer com que o elemento anterior a ele passe a apontar para o elemento seguinte, e então podemos libertar o elemento que queremos retirar. Devemos notar que, no segundo caso, precisamos do apontador para o elemento anterior para podermos acertar o encadeamento da lista.



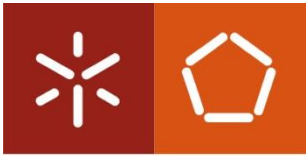


Listas Ligadas Simples

Universidade do Minho

- Remover um elemento

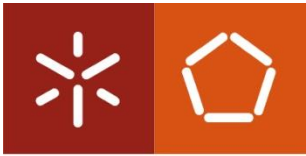
```
int removerElemento(node_t **lista, data_t valorProcurado){
node_t *aux, *ant;
int encontrou=0;
aux = *lista;
ant = *lista;
/* Percorre a lista até encontrar o elemento (nó) procurado */
while (encontrou == 0 && aux != NULL){
    if (igual(aux->data,valorProcurado)) /* função igual a implementar */
        encontrou = 1;
    else{
        ant = aux;
        aux=aux->next;
    }
}
```



Listas Ligadas Simples

Universidade do Minho

```
if (encontrou){
    /* Verifica se o elemento (no') encontrado e' o primeiro da lista */
        if (ant == aux && aux == *lista){
            *lista = aux->next;
        }
    /*Caso o elemento(nó) encontrado nao seja o primeiro da lista*/
    else{
        ant->next=aux->next;
    }
    /* Liberta a memoria alocada */
    free(aux);
    aux = NULL;
    return 1;
}
/*Retorna zero indicando que o elemento(no) nao foi encontrado*/
return 0;
}
```

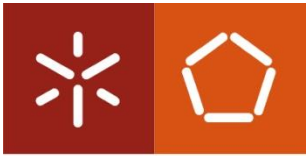


Listas Ligadas Simples

Universidade do Minho

- Travesia – $O(n)$:
 - percorrer todos elementos da lista
- Algoritmo típico:

```
Node *aux=first;
while(aux!=NULL) {
    // processamento
    aux = aux->next;
}
```



Universidade do Minho

Exercícios

- Criar uma estrutura Aluno, com número mecanográfico e nome. A estrutura deve ter um apontador “next” para suportar operações com listas ligadas.